

Agent Systems 研究型进阶专题 A · 第 6 讲

Shared Blind Spot 到底从哪里来？——第一次系统机制消融实验

本讲定位：第 5 讲发现的是“相关结构”：Agent 与 Verifier 可能在同一批困难样本上共同失败。第 6 讲开始做机制实验：固定同一批 proposal，只改变 Reviewer 结构，区分模型同源性、candidate anchoring 与共同证据瓶颈。

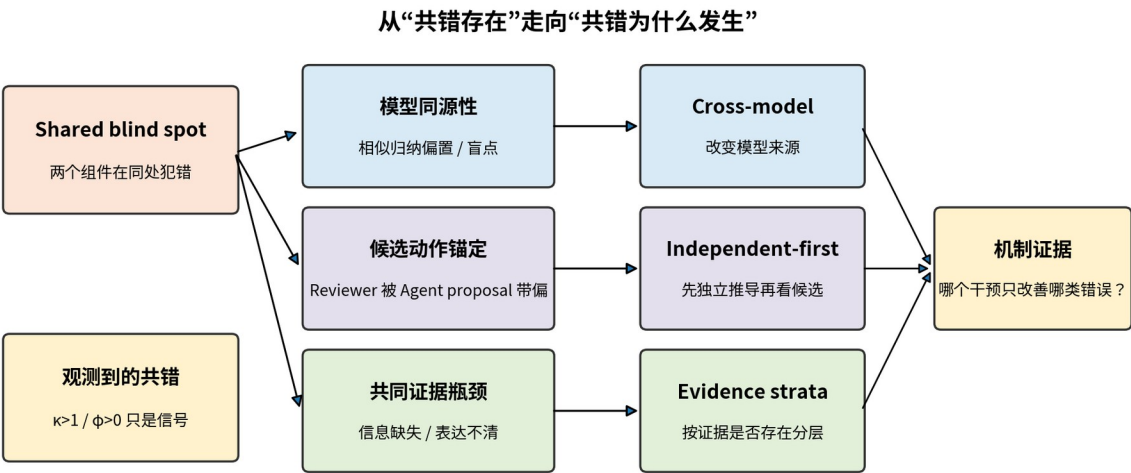


图 1 从共错统计信号到可检验的机制假设

1. 为什么 $\kappa > 1$ 还远远不够？

上一讲的 κ (co-failure ratio) 和 ϕ (phi coefficient) 告诉我们：Agent 错的时候，Verifier 是否也更容易错。但“错误相关”不是“错误原因”。两个组件一起犯错，至少可能来自三种完全不同的机制。

候选机制	通俗解释	如果是真的，我们应观察到什么
Model homogeneity / 模型同源性	两个组件来自相似模型家族，可能共享归纳偏置和盲点	换成不同模型家族后，共错下降
Candidate anchoring / 候选动作锚定	Reviewer 一上来就看到 Agent 的候选动作，被它带偏	先独立推导要求、再看 candidate 后，漏检下降
Shared evidence bottleneck / 共同证据瓶颈	两者看到的信息本身就缺失或不够明确	无论换模型还是去锚定，同一类样本仍持续失败

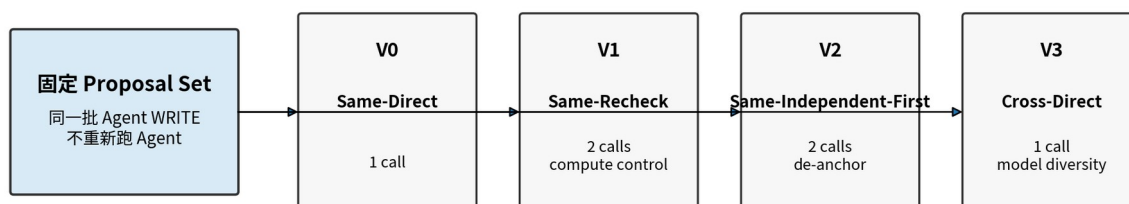
科研原则：Observation → Mechanism Hypothesis → Minimal Intervention。不要从“相关”直接写成“因果”。

2. 本讲最关键的实验技巧：Fixed Proposal Set

如果我们每比较一个 Reviewer 都重新跑一次 Agent，Agent 自己的随机性、用户模拟器随机性、tool path 和后续 trajectory 都可能变化。这样很难知道结果变化到底来自 Reviewer，还是输入已经换了。

所以本讲把第 5 讲已经产生的 WRITE proposals 固定下来。每一个 proposal 的上下文、candidate tool、arguments、人工 VALID/INVALID 标签都不变。然后让不同 Reviewer 条件审查同一条记录。

同一输入、同一人工标签；只改变 Reviewer 结构



配对比较 > 重新采样：每个 proposal 都在所有条件下被再次判断

图 2 固定 proposal 的离线配对实验：所有 Reviewer 看同一批输入

通俗比喻：像把同一套试卷复印四份给四种阅卷流程，而不是让四个班级各自随机出一套题。

- 优势 1：天然形成 paired comparison（配对比较），同一 proposal 可以直接看 verdict 有没有翻转。
- 优势 2：不用重复支付 Agent/User/Environment 的运行成本。
- 优势 3：方便做真正的机制归因：输入不变，只改变 Reviewer 结构。
- 限制：这是离线审计，不能直接等价成在线系统最终成功率；后续仍需在线 intervention 验证。

3. 四个 Reviewer 条件到底在控制什么？

Variant	调用次数	核心操作	它在检验什么
V0 Same-Direct	1	与 Agent 同模型家族，直接看 candidate 并判断	基线共盲点
V1 Same-Recheck	2	第一次直接判断；第二次仍看到 candidate，再认真复核	额外计算 / 第二次机会
V2 Same-Independent-First	2	第一阶段看不到 candidate，只从历史推导 WRITE 必须满足的条件；第二阶段再比较 candidate	candidate anchoring / error propagation
V3 Cross-Direct	1	换成不同模型家族，其他输入与 direct prompt 尽量一致	model homogeneity / diversity

机制消融不是“哪个分数最高”，而是“哪一个最小改变只影响哪类错误”

比较	控制了什么	主要想检验	仍然存在的混杂
V1 vs V0	同模型；V1 多一次复核	单纯增加计算是否有帮助	第二次调用仍可能有随机性
V2 vs V1	同模型、同为2次调用	先独立推导能否降低 candidate anchoring	提示结构也改变
V3 vs V0	同为1次直接审查	模型异质性是否降低 shared blind spot	cross model 能力可能更强/更弱
Evidence strata	不改 Reviewer	共错是否集中在证据缺失类	人工机制标签质量

图 3 每个对比的控制变量与剩余混杂

3.1 为什么不能只比较 V2 和 V0？

因为 V2 用了两次 LLM 调用，而 V0 只用一次。如果 V2 更好，可能只是“多算了一遍”。所以加入 V1 Same-Recheck：它同样用两次调用，但两次都直接看到 candidate。

关键对比： V2 vs V1 比 V2 vs V0 更接近“去锚定”的机制检验，因为两者调用次数相同。

3.2 为什么 V3 仍然不能完全明“模型同源性”？

因为 Cross model 可能本身就更强或更弱。即使 V3 漏检减少，也可能是能力差异，而不是“异质性”本身。因此真实论文阶段应尽量选能力、成本相近的模型，或者加入更多模型对并报告边界。

常见误区：“换 Claude 以后更好，所以异质性一定导致 decorrelation。”——这把模型能力差异和模型来源差异混在了一起。

4. Independent-First 到底为什么能测 Candidate Anchoring？

普通 Direct Reviewer 的顺序是：先看到 Agent 提出的动作，再思考它是否合理。这个顺序可能让 candidate 自己变成一个强烈的“暗示”。如果 Agent 错得很自信，Reviewer 可能顺着这个动作寻找支持它的理由。

Independent-First 把流程拆成两步：第一步完全隐藏 candidate，只让 Reviewer 根据 policy 和可见历史列出“任何合理 WRITE 必须满足的前置条件”；第二步才把 candidate 暴露出来，让它做匹配。

Phase 1 (看不到 candidate)
Observable history -> derive requirements

Phase 2 (才看到 candidate)
requirements + candidate -> ALLOW / RECONSIDER

注意：这不是让模型输出私有思维链。第一阶段只输出可审计的“要求清单 / checklist”，它本身就是实验中可保存的中间变量。

5. Shared Evidence Bottleneck 怎么测？

这一机制不能只靠换模型。需要人工给 INVALID proposal 再加一层机制标签：执行正确判断所需的关键证据，在 Agent 当时可见的 trace 中是否已经存在。

needed_evidence_present	含义	典型情况
YES	关键信息已经在用户消息、tool result 或 policy 中出现	模型读漏、关系推理错、参数引用错
NO	系统还没有获取执行 WRITE 所需的关键事实	应该先 READ/询问，但 Agent 与 Verifier 都贸然放行
UNCERTAIN	你无法可靠判断	先排除出机制统计，保留为审计案例

如果 needed_evidence_present=NO 的 INVALID proposals 在 V0/V1/V2/V3 下都持续被漏过，而 evidence=YES 的样本能被 Cross 或 Independent-First 明显救回，这就是一种很有价值的机制分层。

6. 分析时不要只看总体 Accuracy

对 verifier 最重要的不是普通 accuracy，而是两种方向不同的错误：危险漏放（INVALID + ALLOW）和误拦截（VALID + RECONSIDER）。它们的系统含义完全不同。

指标	公式/统计对象	为什么重要
Dangerous miss rate	INVALID 中被 ALLOW 的比例	Agent 已错，Verifier 又漏过
False block rate	VALID 中被 RECONSIDER 的比例	过度谨慎，可能损害可用性
κ co-failure ratio	实际共同失败 / 独立预期	与第 5 讲保持连续
ϕ error correlation	Agent 错误与 Verifier 错误的二元相关	方向性相关信号
Paired miss→catch	V0 漏过、Variant 成功拦住的同一 proposal 数量	最直接的“救回”证据
Paired catch→miss	V0 能拦住、Variant 反而漏过	防止只报好消息

6.1 Paired flip 为什么特别有价值？

假设总体 miss rate 从 40% 变成 30%。单看这个数字，你不知道是哪些任务变了。但配对后可以看到：10 个原本 miss 的 proposal 被救回，同时又新增 5 个 miss，净改善只有 5。这比“平均分上涨 10 个点”更接近机制证据。

7. 教学 Mock：它要教你看什么，不代表真实模型

重要声明：本资料包的 teaching mock 是人为控制的数据，只用于验证分析代码和理解实验逻辑，不代表任何真实 GPT / Claude / Gemini / DeepSeek / Kimi 模型表现。

教学 Mock 固定 48 个 proposals，其中 20 个 INVALID、28 个 VALID。我们刻意让不同 Variant 对不同 error family 呈现不同改善，用来测试脚本能否恢复这些结构。真实实验中你不能预设这种结果。

教学设定（非真实结果）	Same-Direct	Same-Recheck	Independent-First	Cross-Direct
INVALID dangerous misses	13/20	11/20	8/20	8/20
其中 argument_grounding misses	4/6	3/6	1/6	2/6
其中 policy_misread misses	3/6	3/6	2/6	1/6
VALID false blocks	4/28	4/28	5/28	5/28

你应该读出的不是“Independent 最好”这种排名，而是：在这个受控 Mock 里，Independent-First 特别改善 argument grounding / anchoring 类错误；Cross-Direct 特别改善 policy_misread 类错误；missing_evidence 类仍然顽固。脚本若能恢复这个结构，说明分析流水线正确。

8. 结果模式应该怎样解释？

结果解释必须保留替代解释：不要从一个对比直接跳成“机制已证明”

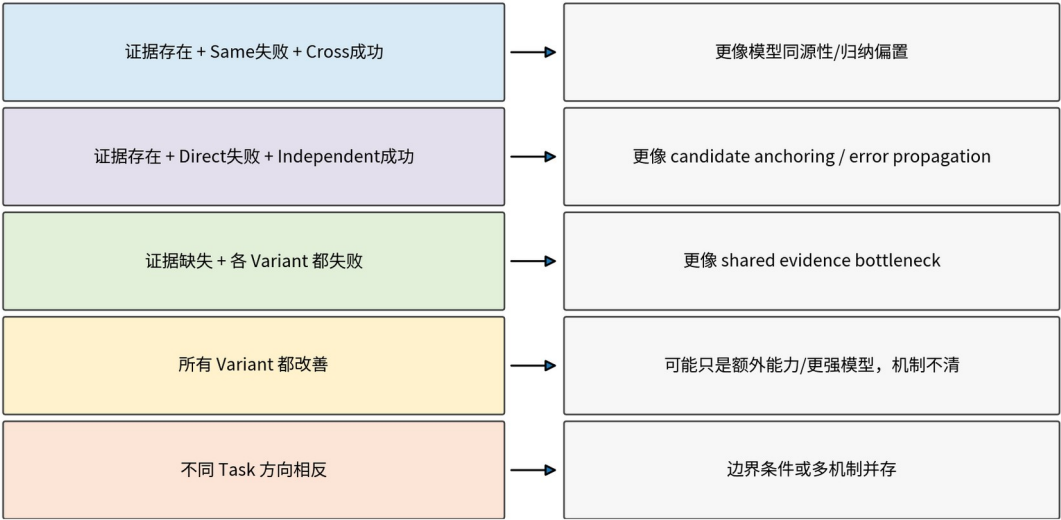


图 4 常见结果模式与谨慎的机制解释

科研边界：“支持某机制” ≠ “机制已被证明”。你还需要跨 task、跨模型、跨 seed 的重复，以及能排除主要替代解释的后续实验。

9. 真实实验最小执行闭环

1. 从第 5 讲拿到已经人工标过 VALID / INVALID 的 A05 shadow audit 结果。
2. 运行 build_fixed_proposals.py, 把同一批 proposals 固定成 JSONL, 并生成机制标签 CSV。
3. 先人工补 INVALID proposal 的 needed_evidence_present、error_family、anchoring_plausible。
4. 配置 exact model IDs。Same model 尽量与 Agent 完全一致；Cross model 选不同模型家族且能力/价格尽量可比。
5. 先 `--limit 8` 或 `--limit 12` 做 API smoke test。
6. 确认解析、成本和日志正常后, 再扩展到 30~50 proposals。
7. 运行 analyze_mechanism_ablation.py, 先看 variant_summary, 再看 paired_invalid_flips。
8. 精读三类 Trace: Direct miss→Independent catch; Direct miss→Cross catch; 所有 Variant 都 miss。
9. 把观察写成可证伪 Hypothesis Card, 而不是直接设计“大方法”。

```
python src/build_fixed_proposals.py \  
  --results /path/to/a05/results.json \  
  --annotations /path/to/a05/write_annotation.csv \  
  --out-jsonl data/fixed_proposals.jsonl \  
  --out-mechanism-csv data/mechanism_annotations.csv  
  
python src/run_verifier_variants.py data/fixed_proposals.jsonl \  
  --config configs/a06_variants.json \  
  --out results/verifier_outputs.jsonl --limit 12  
  
python src/analyze_mechanism_ablation.py \  
  --proposals data/fixed_proposals.jsonl \  
  --verifier-outputs results/verifier_outputs.jsonl \  
  --mechanism-labels data/mechanism_annotations.csv \  
  --out-dir results/a06_analysis
```

10. 本讲真正应该形成的科研直觉

- 复杂 AI 系统里, “加一个组件”不是自动提高可靠性; 关键是组件错误是否互补。
- 共错统计只是现象, 机制需要最小干预来区分。
- 固定输入、配对比较通常比重新采样更适合机制实验。
- Compute-matched control 能防止把“多算了一遍”误写成“新机制有效”。
- 异质性可能 decorrelate error, 但模型能力差异是必须正视的混杂。
- 如果所有结构性干预都救不回来, 输入/证据瓶颈反而可能是更重要的系统问题。

与 AI Systems Science 的连接: 研究对象已经从“模型对不对”变成: 有限、随机、会犯错的组件如何因为结构、信息和交互形成系统级错误模式。

11. 官方实现依据（截至 2026-09-29）

- τ -bench / τ^2 -bench 仓库当前最新 GitHub release: v1.0.1 (2026-07-22)。
- 官方 Agent Developer Guide 支持自定义 HalfDuplexAgent、factory registration 和 programmatic runner。
- ToolKit 中显式区分 READ / WRITE / THINK / GENERIC, WRITE 默认 mutates_state=True; 这也是前几讲以 WRITE proposal 为审计对象的实现依据。
- 本讲的 fixed-proposal offline audit 是我们为了机制实验设计的研究层方法, 不是官方 leaderboard metric。

官方参考: github.com/sierra-research/tau2-bench (release v1.0.1; docs/agent、runner、environment/toolkit)。